Base restaurant_db — service restaurant

Hôtel & Restaurant — Dictionnaire des données V1 (MPD Merise)

Ce document explique **chaque table et chaque attribut** des 7 bases de données : ce qu'il contient, **pourquoi** il existe et **quelle logique** l'utilise. Il correspond au **MPD** (modèle physique) de Merise : les fichiers sources sont `services/<service>/prisma/schema.prisma` et les migrations SQL dans `services/<service>/prisma/migrations/`.

---

0. Les conventions (à lire en premier, elles valent pour toutes les tables)

Convention	Exemple	Pourquoi
Clé primaire `id` en UUID	`78072c82-869c-…`	Généré sans demander à la base, impossible à deviner (on ne peut pas essayer `id=1, 2, 3…` pour lire les données des autres), et unique **entre services** : un `reservationId` créé par hebergement peut voyager dans un événement sans collision.
Nom en camelCase dans le code, snake_case en base	`tarifNuit` ↔ `tarif_nuit` (`@map`)	Convention de TypeScript d'un côté, de PostgreSQL de l'autre. `@@map("chambre")` donne le nom de la table.
Montants en `Int` (entier), en Ariary	`85000`	L'Ariary n'a pas de centimes en pratique, et un entier n'a **jamais d'erreur d'arrondi** (contrairement à `0.1 + 0.2` en virgule flottante).
`createdAt` / `updatedAt`	`@default(now())` / `@updatedAt`	Traçabilité : quand la ligne a été créée, quand elle a changé pour la dernière fois. Prisma remplit `updatedAt` tout seul.
`Timestamptz` pour un instant, **`Date`** pour un jour	`checkInLe` vs `dateArrivee`	Un instant (« 14 h 32 à Tana ») a besoin du fuseau horaire ; une date de séjour (« le 15 octobre ») n'a pas d'heure. Mélanger les deux crée des décalages d'un jour.
`Json` pour les textes traduits (`TexteTraduit`)	`{"fr": "Chambre standard", "mg": "Efitra tsotra"}`	Une seule colonne pour toutes les langues. Ajouter l'anglais = ajouter une clé `en`, sans migration.
« sans FK » + « snapshot »	`clientId` (sans FK) + `clientNom` (snapshot)	**Règle de la SOA** : une base n'a pas de clé étrangère vers la base d'un autre service (ce sont des bases séparées). On garde l'**identifiant** (pour retrouver la donnée) et une **copie figée** des champs utiles (pour afficher sans appeler l'autre service, et pour garder la valeur **au moment de l'action** : si le client change de nom, sa facture garde l'ancien).
`version`	`version Int @default(1)`	**Verrou optimiste** (commande, reservation, tache_menage) : chaque modification fait `version + 1` et exige la version lue (`where: { id, version }`). Si deux personnes modifient en même temps, la seconde modifie 0 ligne et reçoit un conflit (409) au lieu d'écraser la première (« mise à jour perdue »). Pour `chambre`, l'incrément sert à **verrouiller la ligne** pendant le check-in (voir cette table).
Enum	`statut StatutReservation`	Liste fermée de valeurs, vérifiée par la base : une faute de frappe (`"CONFIMEE"`) est impossible.
Contraintes `CHECK` / `EXCLUDE` dans les migrations	`CHECK (quantite >= 0)`	Prisma ne sait pas les écrire dans `schema.prisma` : elles sont ajoutées **à la main** dans le SQL de migration. C'est la **dernière ligne de défense** : même un bug dans le code ne peut pas les violer.

Les 3 tables techniques (présentes dans plusieurs services)

`outbox_evenement` — patron *Transactional Outbox* (hebergement, facturation, menage, stock, restaurant, comptabilite)

Attribut	Type	Description et logique
`id`	uuid	Sert aussi d'**identifiant de l'événement** (les consommateurs s'en servent pour l'idempotence).
`type`	texte	Nom de l'événement : `reservation.creee`, `menage.termine`… (liste dans `packages/commun/src/evenements.ts`).
`payload`	json	Le contenu de l'événement, validé par un schéma Zod.
`createdAt`	instant	Ordre de publication.
`publieLe`	instant, **nul tant que non publié**	Le relais lit les lignes où `publieLe` est nul, les envoie dans Redis/BullMQ, puis remplit la date. Index `(publieLe, createdAt)` pour trouver vite les événements en attente.

Pourquoi : l'événement est écrit dans **la même transaction** que la donnée métier. Soit les deux sont enregistrés, soit aucun. Si Redis est en panne, l'événement attend ici et part plus tard : il n'est jamais perdu.

`evenement_traite` — consommateur idempotent

Attribut	Type	Description et logique
`evenementId`	uuid, **clé primaire**	L'id de l'événement reçu. Inséré **avant** de traiter, dans la même transaction que l'effet. Si le même événement arrive deux fois, l'insertion viole la clé primaire, la transaction est annulée : **aucun double effet** (jamais deux tâches de ménage pour le même séjour). Utilisé par la fonction `traiterUneFois` (`src/outbox.ts` de chaque service). Exception : stock s'appuie sur la clé unique métier `(referenceId, articleId)` de `mouvement`, une transaction par article.
`traiteLe`	instant	Quand il a été traité.

`compteur_<x>` (`compteur_reservation`, `compteur_facture`, `compteur_commande`)

Attribut	Type	Description et logique
`annee`	entier, **clé primaire**	Un compteur par année : la numérotation repart à 1 chaque 1er janvier.
`dernierNumero`	entier	Dernier numéro donné. Un `upsert` + `increment` **atomique** garantit que deux créations simultanées n'obtiennent jamais le même numéro → `RES-2026-000042`, `FAC-2026-000123`, `CMD-2026-000045`. Pour les factures, la numérotation **sans trou** est une obligation comptable.

6. Base `restaurant_db` — service restaurant

Table `menu`

Attribut	Type	Description et logique
`id`	uuid	
`nom`	json (traduit)	« Carte malgache ».

Attribut	Type	Description et logique
`actif`	booléen (défaut faux)	**Publié** ou non. Seuls les menus actifs apparaissent sur le site (c'est la Communication qui publie).

Table `plat`

Attribut	Type	Description et logique
`id`	uuid	
`menuId`	uuid, FK → menu	Le menu qui contient le plat.
`nom`, `description`	json (traduit)	
`categorie`	enum `CategoriePlat`	`ENTREE`, `PLAT`, `DESSERT`, `BOISSON` : l'ordre d'affichage de la carte.
`prix`	entier (Ar), **CHECK > 0**	**Le seul prix qui compte** : le serveur le relit à chaque commande, le navigateur n'envoie que l'id du plat et la quantité.
`imageUrl`	texte	Prévu, pas encore utilisé en V1.
`tempsPreparationMin`	entier (défaut 15)	Pour estimer l'attente.
`disponible`	booléen	Faux = « Épuisé » : disparaît de la prise de commande, barré sur le site.
`disponibleADistance`	booléen	Prévu pour la commande à distance (E5), pas utilisé en V1.

À venir (F7 / E2) : tables `ingredient` et `recette_plat` (plat × ingrédient × quantité).

Table `commande`

Attribut	Type	Description et logique
`id`	uuid	
`reference`	texte, **unique**	`CMD-2026-000045` (`compteur_commande`).
`canal`	enum `CanalCommande`	`SUR_PLACE`, `A_EMPORTER`, `LIVRAISON` (prévu).
`tableNumero`	texte	Pour le service en salle.
`clientId`	uuid (auth), sans FK	Si un client connecté commande (prévu E5).
`clientNom`, `clientTelephone`	texte (snapshots)	
`heureSouhaitee`	instant	Prévu pour la commande à distance.
`statut`	enum `StatutCommande`	**Préparation** : `RECUE` → `EN_PREPARATION` → `PRETE` → `REMISE`, ou `ANNULEE`.
`statutPaiement`	enum `StatutPaiementCommande`	**Paiement**, indépendant de la préparation : `NON_PAYEE`, `PAYEE`, `IMPUTEE_CHAMBRE` (ajoutée à la facture du séjour), `A_LA_LIVRAISON` (prévu). Deux statuts séparés car on peut servir avant de payer, ou payer avant d'être servi.
`reservationId`, `chambreNumero`	uuid sans FK + snapshot	Remplis à l'**imputation à une chambre** : le séjour en cours dans cette chambre. Déclenche l'événement `consommation.imputee` vers facturation.

Attribut	Type	Description et logique
`montantTotal`	entier, CHECK ≥ 0	Somme des lignes, calculée par le serveur.
`fraisLivraison`	entier, CHECK ≥ 0	Prévu (V3).
`serviPar`	uuid (auth), sans FK	Le serveur qui a pris la commande.
`version`	entier	Verrou optimiste (deux serveurs changent le statut en même temps).

Table `ligne_commande`

Attribut	Type	Description et logique
`id`	uuid	
`commandeId`	uuid, FK → commande (cascade)	
`platId`	uuid, FK → plat	
`platNom`	texte (**snapshot**)	Le nom au moment de la commande, dans la langue de la commande.
`prixUnitaire`	entier (**snapshot**), CHECK > 0	**Un changement de prix demain ne modifie pas les commandes passées.**
`quantite`	entier, CHECK > 0	

9. Ce qui changera avec le SaaS (Phase 2)

- **Toutes** les tables métier recevront `hotelId` (sauf les tables propres au client global).
- Les uniques deviendront **par hôtel** : `(hotelId, numero)` pour les chambres, `(hotelId, code)` pour les catégories et les articles, compteurs `(hotelId, annee)`.
- `utilisateur` deviendra **global** ; une nouvelle table `membre (utilisateurId, hotelId, role)` portera les rôles du personnel.
- `outbox_evenement` et les événements porteront le `hotelId`.
- Les politiques **RLS** de PostgreSQL filtreront par `hotelId` (personnel) ou par `clientId` (client).